

Today's Progress - Full Work Log

Parking Zoning Foundation - Damanat PMS Video Analytics

Date: 2026-06-26 Author: Mohamed Henaish

Branch: feat/single-camera-ownership-and-anpr-coalescing

Goal of the day: prepare the codebase for parking ZONING - splitting B1/B2 into areas (aisles + ramps) so the vehicle ReID matcher only searches cars that can physically be present (cheaper + more accurate). Everything below was built, tested, verified against the live MSSQL server, and committed (3 commits). Behaviour is unchanged until areas are configured.

0. Floor label cleanup

- Renamed the ground-floor label 'Ground Floor' -> 'Ground' on CAM-01/02 (config.yaml + config.example.yaml) for consistency with B1/B2.

1. Config & data model

- Added CameraEntry.area (each camera carries one home area).
- Added AreaEntry + AppConfig.areas (area_id, name, floor, capacity, adjacency) and parsing of an 'areas:' block.
- Helper methods: area_for_camera(), cameras_in_area(), area_by_id(), adjacency_for().
- config.yaml / config.example.yaml: defined 8 areas (B1-A/B/C + B1-RAMP, B2-A/B/C + B2-RAMP) and added 'area:' to every B1/B2 camera.
- VehicleSession gained area-state fields: current_area, area_state, area_entered_at, departed_from_area.

2. Areas read from the database at runtime

- New parking_areas table (owned by this repo, ORM src/model/parking_area.py).
- ensure_areas_initialized() seeds the table from config.yaml when empty; sync_areas_from_db() makes the DB the source of truth at runtime.
- Wired into main.py next to the existing Config seed/sync flow.
 - . config.yaml is only the first-run seed; after that the database wins.

3. Bounded (per-area) matching

- VehicleRegistry._area_sessions: an area_id -> {car} index, maintained on enter / move / exit.
- match_global_session(area_id=...) restricts the candidate pool to in-area cars + the adjacent-area hand-off pool.
- Fully backward-compatible: with no area it behaves exactly as before (searches all cars).

4. Zoning package (src/zoning/)

- AreaRegistry - working: camera->area, capacity, adjacency, transit times.
- CrossAreaHandoffMatcher - builds the candidate set of cars that just left an adjacent area within the transit window.
- BoundaryCrossingDetector - point-in-polygon crossing (reuses existing geometry); inert until boundaries are drawn.
- AreaStateMachine - IN_AREA -> DEPARTING -> IN_TRANSIT -> ARRIVING; drops a departing car from its old area; bidirectional ramps.
- IntraAreaFusion - one identity per car across an area's cameras (stub, wired for later tuning).

5. Boundaries (area-to-area gates)

- Stored in their OWN 'boundaries' table (not on parking_slots) - ORM src/model/boundary.py.
- A boundary is a pairwise gate (area_from -> area_to). A junction where 3+ areas meet = several gates.
- Bidirectional: managers drive ramps both ways, so the crossing direction is resolved from the car's current area (verified by test).
- parking_service: load_camera_boundaries() + sync_camera_boundaries(); load_camera_slots() now returns (parking, special, roi, boundaries).

6. Area naming + ramp direction correction

- Renamed B1 areas B1-D/E/F -> B1-A/B/C so each floor's aisles start at A.
- Fixed the B1 ramp label: 'B1 Ramp (down to B2)' - B1 leads DOWN to B2.
- Applied to config.yaml, config.example.yaml AND reseeded the live MSSQL parking_areas table.

7. Drawing tool (draw_slots.py)

- New 'b' BOUNDARY mode: draw a band across the lane, then prompts for area_from / area_to.
- Area-id VALIDATION against the configured areas (typo re-prompts; blank cancels; from==to rejected).
- Bug fix: in-progress click points were not drawn in BOUNDARY mode ('can't click'); now rendered (magenta), with Points counter + undo.
- . Slots save to parking_slots; boundaries save to the boundaries table.

8. Entry ReID - line-crossing endpoint (new integration)

Agreed with the other-server team: after the ANPR plate event, the external line-crossing detector posts the CAM-03 entry frame; we crop the entering car and use it for ReID.

```
POST /api/line-crossing (JSON, like the ANPR image)
{ image_base64, camera_id='CAM-03', plate? }
-> detect+crop entering car -> seed ReID -> bind to pending plate
```

- Server-side vehicle crop (the frame also has parked cars) via the YOLO detector; sender posts the FULL frame.
- Also fixed a pre-existing Windows-console UnicodeEncodeError in the ANPR handlers.

9. Admin tools & scripts

- tools/manage_areas.py - edit the zoning tables in the live MSSQL DB: list, boundaries, push (apply config.yaml), set, delete, reseed.
- tools/add_camera_area_column.py - add the cameras.area column (+ optional populate); normalizes Cam_03 vs CAM-03 id forms.

10. Key decisions made today

- Areas = aisles; the ramp is its own area; Ground floor excluded.
- Areas live in the DB and are read at runtime (config.yaml = seed only).
- Boundaries in their own table; pairwise; bidirectional.
- Ground entry/exit handled by the existing ANPR / B1-entrance flow.
- The API Gateway 'zone' (e.g. 'B1 Parking', 6 cams) maps to our FLOOR level; our 'area' is a finer layer inside it.

11. Quality, tests & review

- New tests/test_zoning_foundation.py - 14 tests (config, AreaRegistry, bounded matching, hand-off, boundary geometry, state machine, DB round-trips).
- Existing regression suites still pass (single-camera ownership, confirmation timeline).
- Codex code review run; both P2 findings fixed (4-tuple callers; departing car dropped from old area).
- Verified live against the MSSQL server end to end.

12. Documentation

- README: new 'Zoning (Parking Areas)' section - the model, a from-zero setup guide, DB editing, the line-crossing endpoint, and the tools.

13. Commits (branch, not yet pushed)

```
f4eb800 feat(zoning): parking-area foundation + line-crossing ReID
628bfdb feat(tools): add_camera_area_column.py
c23b647 fix(draw_slots): render in-progress points in BOUNDARY mode
```

14. Open items / next

- Finalize the real camera-to-area assignments (placeholders in place).
- Draw the ramp boundary bands on the camera views.
- Wire the engine to read camera.area from the DB (column added; engine still reads config.yaml).
- Tune the AI-Dev pieces (hand-off thresholds, multi-view fusion) when the new cameras are accessible.
- Review & merge the branch.